Streaming

FFmpeg can basically stream through one of two ways: It either streams to a some "other server", which re-streams for it to multiple clients, or it can stream via UDP/TCP directly to some single destination receiver, or alternatively directly to a multicast destination. Theoretically you might be able to send to multiple receivers via multiple outputs but there is no built-in full blown server.

Servers which can receive from FFmpeg (to restream to multiple clients) include ffserver (linux only, though with cygwin it might work on windows), or Wowza Media Server, or Flash Media Server, Red5, or various others. Even VLC can pick up the stream from ffmpeg, then redistribute it, acting as a server. Since FFmpeg is at times more efficient than VLC at doing the raw encoding, this can be a useful option compared to doing both transcoding and streaming in VLC. Nginx also has an rtmp redistribution plugin, as does apache etc. and there is probably more out there for apache, etc.. You can also live stream to online redistribution servers like own3d.tv or justin.tv (for instance streaming your desktop). Also any rtmp server will most likely work to receive streams from FFmpeg (these typically require you to setup a running instance on a server).

Contents

The -re flag
Setting
Latency
CPU usage / File size
Streaming a simple RTP audio stream from FFmpeg
Streaming a ZMQ audio stream from FFmpeg
Codecs
HTTP Live Streaming and Streaming with multiple bitrates
Saving a file and Streaming at the same time
Transcoding / repeating
Adjusting bitrate based on line conditions
Troubleshooting Streaming
Point to point streaming
External links

The -re flag

The FFmpeg's "-re" flag means to "Read input at native frame rate. Mainly used to simulate a grab device." i.e. if you wanted to stream a video file, then you would want to use this, otherwise it might stream it too fast (it attempts to stream at line speed by default). My guess is you typically don't want to use this flag when streaming from a live device, ever.

Setting

Here is what another person once did for broadcast:

```
ffmpeg -f dshow -i video="Virtual-Camera" -preset ultrafast -vcodec libx264 -tune zerolatency -b 900k -f mp
```

And here is what another person did:

```
ffmpeg -f dshow -i video="screen-capture-recorder":audio="Stereo Mix (IDT High Definition" \
-vcodec libx264 -preset ultrafast -tune zerolatency -r 10 -async 1 -acodec libmp3lame -ab 24k -ar 22050 -bs
-maxrate 750k -bufsize 3000k -f mpegts udp://192.168.5.215:48550
```

NB that they also (for directshow devices) had to adjust the rtbufsize in that example.

You can see a description of what some of these means, (for example bufsize, bitrate settings) in the Encode/H.264.

Here's how one guy broadcast a live stream (in this instance a Capture/Desktop#Windows? screen capture device):

```
$ ffmpeg -y -loglevel warning -f dshow -i video="screen-capture-recorder" -vf crop=690:388:136:0 -r 30 -s 9
```

with a custom FFmpeg preset (libx264-my_ffpreset.ffpreset) in this case:

```
coder=1
flags2=+wpred+dct8x8
level=31
maxrate=1200000
bufsize=200000
```

```
wpredp=0
g=60
refs=1
subq=3
trellis=0
bf=0
rc_lookahead=0
```

Here is how you stream to twitch.tv or similar services (rtmp protocol), using ffmpeg 1.0 or ffmpeg-git (tested on 2012-11-12), this is also for pulseaudio users: Example 1, no sound:

```
ffmpeg -f x11grab -s 1920x1200 -framerate 15 -i :0.0 -c:v libx264 -preset fast -pix_fmt yuv420p -s 1280x800
```

Example 2, first screen (on dual screen setup, or if on a single screen):

```
ffmpeg -f x11grab -s 1920x1200 -framerate 15 -i :0.0 -f pulse -ac 2 -i default -c:v libx264 -preset fast -p
```

Example 3, second screen (on dual screen setup):

```
ffmpeg -f x11grab -s 1920x1200 -framerate 15 -i :0.0+1920,0 -f pulse -ac 2 -i default -c:v libx264 -preset
```

Latency

You may be able to decrease initial "startup" latency by specifing that I-frames come "more frequently" (or basically always, in the case of x264's zerolatency setting), though this can increase frame size and decrease quality, see here for some more background. Basically for typical x264 streams, it inserts an I-frame every 250 frames. This means that new clients that connect to the stream may have to wait up to 250 frames before they can start receiving the stream (or start with old data). So increasing I-frame frequency (makes the stream larger, but might decrease latency). For real time captures you can also decrease latency of audio in windows dshow by using the dshow audio_buffer_size setting. You can also decrease latency by tuning any broadcast server you are using to minimize latency, and finally by tuning the client that receives the stream to not "cache" any incoming data, which, if it does, increases latency.

Sometimes audio codecs also introduce some latency of their own. You may be able to get less latency by using speex, for example, or opus, in place of libmp3lame.

You will also want to try and decrease latency at the server side, for instance wowza hints.

Also setting -probesize and -analyzeduration to low values may help your stream start up more quickly (it uses these to scan for "streams" in certain muxers, like ts, where some can appears "later", and also to estimate the duration, which, for live streams, the latter you don't need anyway). This should be unneeded by dshow input.

Reducing cacheing at the client side can help, too, for instance mplayer has a "-nocache" option, other players may similarly has some type of pre-playback buffering that is occurring. (The reality is mplayers -benchmark option has much more effect).

Using an encoder that encodes more quickly (or possibly even raw format?) might reduce latency.

You might get less latency by using one of the "point to point" protocols described in this document, at well. You'd lose the benefit of having a server, of course.

NB that a client when it initially starts up may have to wait until the next i-frame to be able to start receiving the stream (ex: if receiving UDP), so the GOP setting (-g) i-frame interval will have an effect on how quickly they can start streaming (i.e. they must receive an i-frame before they start). Setting it to a lower number means it will use more bandwidth, but clients will be able to connect more quickly (the default for x264 is 250--so for 30 fps that means an i-frame only once every 10 seconds or so). So it's a tradeoff if you adjust it. This does not affect actual latency (just connection time) since the client can still display frames very quickly after and once it has received its first i-frame. Also if you're using a lossy transport, like UDP, then an i-frame represents "the next change it will have to repair the stream" if there are problems from packet loss.

You can also (if capturing from a live source) increase frame rate to decrease latency (which affects throughput and also i-frame frequency, of course). This obvious sends packets more frequently, so (with 5 fps, you introduce at least a 0.2s latency, with 10

fps 0.1s latency) but it also helps clients to fill their internal buffers, etc. more quickly.

Note also that using dshow's "rtbufsize" has the unfortunate side effect of sometimes allowing frames to "buffer" while it is waiting on encoding of previous frames, or waiting for them to be sent over the wire. This means that if you use a higher value at all, it can cause/introduce added latency if it ever gets used (but if used, can be helpful for other aspects, like transmitting more frames overall consistently etc. so YMMV). Almost certainly if you set a very large value for this, and then see the "buffer XX% full! dropping!" message, you are introducing latency.

There is also apparently an option -fflags nobuffer which might possibly help, usually for receiving streams reduce latency.

mpv udp://236.0.0.1:2000 --no-cache --untimed --no-demuxer-thread --video-sync=audio --vd-lavc-threads=1

may be useful.

Testing latency

By default, ffplay (as a receiver for testing latency) introduces significant latency of its own, so if you use it for testing (see troubleshooting section) it may not reflect latency accurately. FFplay introduces some video artifacts, also, see notes for it in "troubleshooting streaming" section Also some settings mentioned above like "probesize" might help it start more quickly. Also useful:

```
ffplay -probesize 32 -sync ext INPUT
```

(the sync part tells it to try and stay realtime).

Useful is mplayer with its -benchmark for testing latency (-noaudio and/or -nocache *might* be useful, though I haven't found -nocache to provide any latency benefit it might work for you).

Using the SDL out option while using FFmpeg to receive the stream might also help to view frames with less client side latency: "ffmpeg ... -f sdl <input_here> "window title"" (this works especially well with -fflags nobuffer, though in my tests is still barely more latency than using mplayer -benchmark always). This doesn't have a "drop frames if you run out of cpu" option so it can get quite far behind at times (introduce more latency variably).

Another possibly useful receiving client is "omxplayer -live"

See also "Point to point streaming" section esp. if you use UDP etc.

See also

Here is a list of some other ideas to try (using VBR may help, etc.)

CPU usage / File size

In general, the more CPU you use to compress, the better the output image will be, or the smaller of a file the output will be for the same quality.

Basically, the easiest way to save cpu is to decrease the input frame rate/size, or decrease the output frame rate/size.

Also you could (if capturing from live source), instruct the live source to feed a "smaller stream" (ex: webcam stream 640x480 instead of 1024x1280), or you could set a lower output "output quality" setting (q level), or specify a lower output desired bitrate (see Encode/H.264 for a background). Or try a different output codec, or specify new parameters to your codec (for instance, a different profile or preset for libx264). Specifying $ -threads 0 instructs the encoder to use all available cpu cores, which is the default. You could also resize the input first, before transcoding it, so it's not as large. Applying a smoothing filter like hqdn3d before encoding might help it compress better, yielding smaller files.

You can also set a lower output frame rate to of course decrease cpu usage.

If you're able to live capture in a pixel format that matches your output format (ex: yuv420p output from a webcam, instead of mjpeg), that might help with cpu usage, since it avoids an extra conversion. Using 64-bit instead of 32-bit executables (for

those that have that choice) can result in a slight speedup. If you're able to use -vcodec copy that, of course, uses the least cpu of all options since it just sends the frames verbatim to the output.

Sometimes you can change the "pixel formats" somehow, like using rgb16 instead of rgb24, to save time/space (or yuv420 instead of yuv444 or the like, since 420 stores less information it may compress better and use less bandwidth). This may not affect latency.

Streaming a simple RTP audio stream from FFmpeg

FFmpeg can stream a single stream using the RTP protocol. In order to avoid buffering problems on the other hand, the streaming should be done through the -re option, which means that the stream will be streamed in real-time (i.e. it slows it down to simulate a live streaming source.

For example the following command will generate a signal, and will stream it to the port 1234 on localhost:

```
ffmpeg -re -f lavfi -i aevalsrc="sin(400*2*PI*t)" -ar 8000 -f mulaw -f rtp rtp://127.0.0.1:1234
```

To play the stream with ffplay (which has some caveats, see above), run the command:

```
ffplay rtp://127.0.0.1:1234
```

Note that rtp by default uses UDP, which, for large streams, can cause packet loss. See the "point to point" section in this document for hints if this ever happens to you.

Streaming a ZMQ audio stream from FFmpeg

FFmpeg can stream using the https://en.wikipedia.org/wiki/ZeroMQ if compiled with the `--enable-libzmq` flag at build time. The streaming should also be done utilizing the -re option, for real-time streaming (i.e. it slows it down to simulate a live streaming source. ZMQ is primarily TCP as in this example but other streaming options are possible.

For example the following command will generate a signal, and will stream it to the port 5555 on localhost:

```
ffmpeg -re -hide_banner -f s16le -ar 48000 -ac 2 -i - -f mpegts -acodec libopus -b:a 128k zmq:tcp://127.0.0
```

To play the stream with ffplay, run the command:

```
ffplay -autoexit -nodisp zmq:tcp://${serverAddress}:${streamPort}
```

A secure stream can be accessed with the above ffplay command from a remote client by connecting to the server via an ssh tunnel:

```
autossh -fvN -M0 -L${streamPort}:localhost:${streamPort} ${serverAddress} -p${sshPort}"
```

Where `streamPort` is the port for the stream (also the ssh tunnel port); `serverAddress` and `sshPort` are the server url/ip address and port, respectively. (Adding `-R${returnPort}:localhost:22` allows access from the server back to the client if desirable. `ssh` can be used in place of autossh for a simpler connection if less reliable connection; a VPN tunnel would also suffice.)

Codecs

The most popular streaming codec is probably libx264, though if you're streaming to a device which requires a "crippled" baseline h264 implementation, you can use the x264 "baseline" profile. Some have have argued that the mp4 video codec is better than x264 baseline, because it encodes about as well with less cpu. You may be able to use other codecs, like mpeg2video, or really any other video codec you want, typically, as long as your receiver can decode it, if it suits your needs.

Also note that encoding it to the x264 "baseline" is basically a "compatibility mode" for older iOS devices or the like, see here.

The mpeg4 video codec sometimes also comes "within a few percentage" of the compression of x264 "normal settings", but uses much less cpu to do the encoding. See http://ffmpeg.zeranoe.com/forum/viewtopic.php?f=7&t=631&hilit=mpeg4+libx264+cores&start=10#p2163 for some graphs (which may be slightly outdated). Basically in that particular test it was 54 fps to 58 fps (libx264 faster), and libx264 file was 5.1MB and mpeg4 was 6MB, but mpeg4 used only half as much cpu for its computation, so take it with a grain of salt.

HTTP Live Streaming and Streaming with multiple bitrates

FFmpeg supports splitting files (using "-f segment" for the output, see segment muxer) into time based chunks, useful for HTTP live streaming style file output.

How to stream with several different simultaneous bitrates is described here.

See also http://sonnati.wordpress.com/2012/07/02/ffmpeg-the-swiss-army-knife-of-internet-streaming-part-v

Saving a file and Streaming at the same time

See Creating multiple outputs. Basically, you may only be able to accept from a webcam or some other source from at most one process, in this case you'll need to "split" your output if you want to save it and stream it simultaneously. Streaming and saving simultaneously (and only encoding once) can also save cpu.

Transcoding / repeating

FFmpeg can also receive from "a source" (for instance live or UDP) and then transcode and re-broadcast the stream.

One mailing list user wrote this, quote:

> In my application, I have a server running vlc sucking streams from some cameras, encoding them as MPEG2 streams and sending them to ports 5000 through 5003 on my intermediary server (I'll have to let someone else explain how to set that up, as that was someone else's part of the project). I have another server running Wowza, with an instance named "live". And I've got an intermediate server that sucks in MPEG2 streams coming in on ports 5000 to 5003, transcoding them into mp4 streams with H.264 and AAC codecs, and pushing the transcoded streams on to Wowza.
>
> The command line I use to pull the stream from port 5000, transcode it, and push it is: ffmpeg -i 'udp://localhost:5000?fifo_size=1000000&overrun_nonfatal=1' -crf 30 -preset ultrafast -acodec aac -ar 44100 -ac 2 -b:a 96k -vcodec libx264 -r 25 -b:v 500k -f flv 'rtmp://<wowza server IP>/live/cam0'
>
> -i 'udp://localhost:5000?fifo_size=1000000&overrun_nonfatal=1' tells ffmpeg where to pull the input stream from. The parts after the ? are probably not needed most of the time, but I did need it after all.
>
> -crf 30 sets the Content Rate Factor. That's an x264 argument that tries to keep reasonably consistent video quality, while varying bitrate during more 'complicated' scenes, etc. A value of 30 allows somewhat lower quality and bit rate. See Encode/H.264.
>
> -preset ultrafast as the name implies provides for the fastest possible encoding. If some tradeoff between quality and encode speed, go for the speed. This might be needed if you are going to be transcoding multiple streams on one machine.
>
> -acodec aac sets the audio codec (internal AAC encoder)
>
> -ar 44100 set the audio sample rate
>
> -ac 2 specifies two channels of audio

-b:a 96k sets the audio bit rate

-vcodec libx264 sets the video codec

-r 25 set the frame rate

-b:v 500k set the video bit rate

-f flv says to deliver the output stream in an flv wrapper

'rtmp://<wowza server IP>/live/cam0' is where the transcoded video stream gets pushed to

Adjusting bitrate based on line conditions

FFmpeg doesn't (today) support varying the encoding bitrate based on fluctuating network conditions. It does support outputting in several "different" fixed bitrates, at the same time, however, see "Streaming with multiple bitrates" on this page, which is vaguely related. Also if you are during direct capture from directshow, the input device starts dropping frames when there is congestion, which somewhat simulates a variable outgoing bitrate.

Troubleshooting Streaming

If you get a "black/blank" screen in the client, try sending it yuv422p or yuv420p type input. Some servers get confused if you send them yuv444 input (which is the default for libx264).

NB that when you are testing your stream settings, you may want to test them with both VLC and FFplay, as FFplay sometimes introduces its own artifacts when it is scaled (FFplay uses poor quality default scaling, which can be inaccurate). Don't use ffplay as your baseline for determining quality.

Point to point streaming

If you want to stream "from one computer to another", you could start up a server on one, and then stream from FFmpeg to that server, then have the client connect to that server (server could either be on client or server side computers). Or you could do a point to point type stream, like:

```
ffmpeg -i INPUT -acodec libmp3lame -ar 11025 -f rtp rtp://host:port
```

where host is the receiving IP. Then receive the stream using VLC or ffmpeg from that port (since rtp uses UDP, the receiver can start up any time).

or

```
ffmpeg -i INPUT -f mpegts udp://host:port
```

If you run into packet loss (green frames, tearing/shearing--since UDP is not guaranteed delivery, this can occur) first make sure your FFmpeg is compiled with pthreads support enabled (if it is, then it uses a separate thread to receive from the UDP port, which can cause less packet loss). You can tell that it is by specifying a url like udp://host:post?fifo_size=10000 (if it accepts fifo_size, then you're good to go). Similarly, for mplayer, you can use mplayer ffmpeg://udp://host:port?fifo_size=XXX for possibly better results on the receiving end (mplayer needs a patch first, email rogerdpack@gmail.com for it, once was https://gist.githubusercontent.com/rdp/9075572/raw/002dc1b745c895693fdb160cc9be77ef31f75531/possible_mplayer_udp_fix.diff hackey work around patch, applied with ffmpeg subdir).

Alternatively, increase your buffer size, like mplayer ffmpeg://udp://host:port?buffer_size=10000000 (the default is system dependent and typically far too low for any reasonable buffering. On linux though you can only set it to like 200K max anyway, so this isn't good enough--make sure to use the circular buffer, and that the following works: ffmpeg://udp://host:port?buffer_size=10000000?fifo_size=100000 (the fifo_size should not emit a warning, and implies that you have a secondary thread that collects incoming packets for you if there is no warning).

Another option is to use some transmission type that uses TCP for your transport. (The RTMP protocol, popular in streaming to servers, uses TCP probably for this reason--you just can't use that for point to point streaming).

One option to use TCP is like this:

```
ffmpeg -i INPUT -f mpegts tcp://host:port
```

which I would guess will try and (as a client) establish a connection do that host on that port (assuming it has a server waiting for the incoming connection). You could receive it like this:

```
ffmpeg -i tcp://local_hostname:port?listen
```

(basically, one side needs to specify "listen" and the other needs to not to).

To use with mplayer as a receiver it would be like

```
ffmpeg -i ... -f mpegts "tcp://127.0.0.1:2000"
```

and on the mplayer side

```
mplayer ... ffmpeg://tcp://127.0.0.1:2000?listen
```

(start mplayer first)

Another option is to use RTP (which by default uses UDP) but by specifying it use TCP:

```
ffmpeg -i input -f rtsp -rtsp_transport tcp rtsp://localhost:8888/live.sdp
```

(For meanings of options see official documentation.

Then you may receive it like this (ffplay or ffmpeg):

```
ffplay -rtsp_flags listen rtsp://localhost:8888/live.sdp?tcp
# ending "?tcp" may not be needed -- you will need to start the server up first, before the sending client
```

ffmpeg also has a "listen" option for rtmp so it may be able to receive a "straight" rtmp streams from a single client that way.

With tcp based streams you can probably use any formatting/muxer, but with udp you need to be careful and use a muxer that supports 'connecting anytime' like mpegts.

If you are forced to use udp (for instance you need to broadcast to a multicast port for whatever reason) then you may be able to avoid the packet loss by (sending less data or sending the same frames over and over again so they have a higher chance of being received).

See also the section on i-frames in #Latency.

Final working p2p client, with multicast:

server:

```
ffmpeg -f dshow -framerate 20 -i video=screen-capture-recorder -vf scale=1280:720 -vcodec libx264 -pix_fmt
```

client:

```
mplayer -demuxer +mpegts -framedrop -benchmark ffmpeg://udp://236.0.0.1:2000?fifo_size=100000&buffer_size=1
# (see note above about linux needing fifo_size as well as buffer_size).
```

External links

- Fabio Sonnati's tutorial on streaming with FFmpeg

Last modified on Dec 5, 2024, 1:55:28 PM